

Daily Research report

Subject:

**Implementation of a runtime for
executing computations on
volunteer machines in a
resource-limited environment**

By Ngnapa Ngoule Ashley

18/10/2025

Tasks achieved:

- Reading the Grok article and testing some namespaces working mechanism (following)
- Summary of the previous article on namespaces and cgroups following the methodology
- Testing the functioning of the voluntary application and defining the tasks to be done

Global summary of the article on namespaces and c-groups

- The network isolation : Used principally to isolate the ports numbers, routing tables, firewall rules and network interfaces.

Test: to see exactly which is the network interface to each name space we

- ❖ use the command `ps aux` to display all the PID of running processes in each single name space
- ❖ run the command `ls -l /proc/self/ns/net` to display the network interface in the current namespace
- ❖ use `unshare` to create a new namespace we run the command to display its own network interface we realize that they were really different so the namespace is late the interface

Test: trying to attribute an ip address and some particular firewall rules to a namespace

Test: creating a minimalist namespace with 4 isolations

For this one test, we created a new namespace isolating pids, network, hostname and mount.

the fork option creates a process inside the namespace, for it to be fully fonctionnal.

```
ashley@ashley:~$ sudo unshare --mount --fork --pid --uts --net bash
root@ashley:~# hostname
ashley
root@ashley:~# hostname testnamespace
root@ashley:~# hostname
testnamespace
root@ashley:~# ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.1	0.2	225364	9156	?	Ss	11:25	0:10	/sbin/init
root	2	0.0	0.0	0	0	?	S	11:25	0:00	[kthreadd
root	4	0.0	0.0	0	0	?	I<	11:25	0:00	[kworker/
root	6	0.0	0.0	0	0	?	I<	11:25	0:00	[mm_percp
root	7	0.0	0.0	0	0	?	S	11:25	0:00	[ksoftirq
root	8	0.0	0.0	0	0	?	I	11:25	0:00	[rcu_sche
root	9	0.0	0.0	0	0	?	I	11:25	0:00	[rcu_bh]
root	10	0.0	0.0	0	0	?	S	11:25	0:00	[migratio
root	11	0.0	0.0	0	0	?	S	11:25	0:00	[watchdog
root	12	0.0	0.0	0	0	?	S	11:25	0:00	[crond]

```

root@ashley:~# mount -t proc proc /proc
root@ashley:~# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1  23020  5104 pts/0    S    12:55   0:00 bash
root      15  0.0  0.0  37804  3300 pts/0    R+   12:57   0:00 ps aux
root@ashley:~#

```

Some issues to solve:

- When I close the terminal the name spaces process are all killed and it disappears
- The namespace created using the unshare command first inherit all the previous element of the parent (except the one isolated by the command) and continue as a new system with its own elements so the native content of a namespace depends on the state of the host at which it has been created
- the mount isolation only abstracts the mount information the file created in each space I still visible everywhere

Some essential commands for namespaces management

There are some commands useful for namespaces management.

- *lsns*: Which is the command used to display all the namespaces running in a host

```
ashley@ashley:~$ lsns
      NS  TYPE  NPROCS  PID  USER  COMMAND
4026531835 cgroup    4  1778 ashley /lib/systemd/systemd --user
4026531836 pid      4  1778 ashley /lib/systemd/systemd --user
4026531837 user     4  1778 ashley /lib/systemd/systemd --user
4026531838 uts      4  1778 ashley /lib/systemd/systemd --user
4026531839 ipc      4  1778 ashley /lib/systemd/systemd --user
4026531840 mnt      4  1778 ashley /lib/systemd/systemd --user
4026531993 net      4  1778 ashley /lib/systemd/systemd --user
ashley@ashley:~$
```

- *Unshare*: is the main command used to create a new namespace.

syntax *sudo unshare [options] <command>*.

The options can be *-net; -fork; -uts; -pid; -mount; -user; -ipc; -cgroup; -time* depending on the type of namespace to create.

We can associate more than one option to have a more complete namespace.

eg: *sudo unshare --pid --mount --fork bash*

```
ashley@ashley:~$ sudo unshare --pid --mount --fork bash
[sudo] password for ashley:
root@ashley:~#
```

- *nsenter*: is the command used to enter in an existing namespace. It is most used to manage containers or inspect them without stopping the running processes it contains.

syntax: *sudo nsenter [options] [program [arguments]]*

where options are the same as unshare, and program designates the target namespace's pid.

Appart from that, we can use the option *-all* to entere all the namespaces specified by a pid, or

`-target` to specify the path to the context of the namespace we want to access.

eg:

```
ashley@ashley:~$ sudo nsenter -t 2560 bash
root@ashley:~#
```

- The `ip netns` command, which is a subset of the `ip` command, use especially to manage network namespaces.

Its syntax: `ip netns <command>` and the commands can be `add` (to add a namespace), `delete`, `list`, `exec` (to execute a command in a namespace)

eg:

```
ashley@ashley:~$ sudo ip netns add mynamespace
ashley@ashley:~$ sudo ip netns list
mynamespace
ashley@ashley:~$ listns
listns: command not found
ashley@ashley:~$ lsns
      NS TYPE      NPROCS   PID USER      COMMAND
4026531835 cgroup        4    1778 ashley /lib/systemd/systemd --user
4026531836 pid          4    1778 ashley /lib/systemd/systemd --user
4026531837 user          4    1778 ashley /lib/systemd/systemd --user
4026531838 uts          4    1778 ashley /lib/systemd/systemd --user
4026531839 ipc          4    1778 ashley /lib/systemd/systemd --user
4026531840 mnt          4    1778 ashley /lib/systemd/systemd --user
4026531993 net           4    1778 ashley /lib/systemd/systemd --user
ashley@ashley:~$ sudo ip netns exec mynamespace bash
root@ashley:~# ss
Netid State  Recv-Q  Send-Q      Local Address:Port  LinPeer Address:Port
root@ashley:~# $$
2631: command not found
root@ashley:~#
```

issue: The network namespaces created using the `ip netns` are not displayed with `lsns` cmd because this last command shows only the namespaces attached to at least one running process.

- `setns`

- clone

Global summary of previous article

1. Context and Issues (understandable by a policymaker)

The growing demand for efficient, scalable, and cost-effective computing in cloud infrastructures has driven the evolution of virtualization technologies. Traditional hypervisor-based virtual machines, while secure, consume significant resources. Linux namespaces and c-groups provide an alternative — a lightweight virtualization method that optimizes server utilization, reduces costs, and improves deployment speed. These mechanisms underpin container technologies such as Docker and Kubernetes, which are now essential in digital transformation and cloud-native infrastructures.

2. General Question (understandable by a general director)

How can Linux operating system primitives — namely namespaces and control groups — enable virtualization that balances performance, scalability, and isolation while maintaining operational efficiency?

3. Brief Analysis and State of Knowledge

Virtualization evolved from full virtualization (e.g., VMware, KVM) and paravirtualization toward OS-level virtualization. Containers emerged as a response to the need for faster startup, reduced overhead, and better resource utilization.

While virtual machines isolate hardware through dedicated guest operating systems, containers share the host kernel, providing process-level isolation. This raises subproblems such as:

- How to isolate processes securely using shared kernel resources
- How to allocate and limit resources efficiently
- How to maintain high performance while ensuring adequate isolation

4. Specific Sub-Problem and State of the Art

Sub-problem: How can Linux isolate and manage system resources for multiple concurrent applications with minimal overhead?

State of the Art:

- **Namespaces:** Isolate system resources (process IDs, mounts, networks, IPC, users, etc.) by creating independent kernel views.
- **cgroups (Control Groups):** Manage and limit system resource usage hierarchically across processes (CPU, memory, disk I/O, network).
These mechanisms form the basis for containerization tools like Docker and LXC, improving scalability compared to traditional virtualization.

5. Proposed Model in the Article

The article proposes a model that integrates **Linux namespaces for isolation** and **cgroups for resource control**, forming the architectural core of containers.

Properties:

- Near-native performance (<5% overhead)
- Fine-grained resource management

- Fast deployment and high density per host

Limits:

- Weaker security isolation due to shared kernel
- Vulnerability to kernel-level exploits
- Requires advanced configuration and security hardening

6. Implementation (Architecture, Tools, Experiments)

Architecture:

- Linux kernel (host)
- Namespaces (isolation)
cgroups (resource control)
- Container runtime (Docker/LXC)
- Multiple containers running atop the same kernel

Platform: Linux systems with Docker or LXC.

Simulations/Benchmarks:

- CPU: Containers show only 4% overhead compared to bare metal, while VMs have 15%.
- Memory: Containers 3% overhead vs VMs 20%.
- Network: Containers 4% throughput loss vs VMs 20%.
- Disk I/O: Containers 5% overhead vs VMs 22%.

This demonstrates containers' superior performance and efficiency for microservices and cloud-native applications.

7. General Assessment and Perspectives

The study achieves its initial goal: demonstrating how Linux namespaces and cgroups enable high-performance, lightweight virtualization. Containers represent a key technological shift — bridging the gap between bare-metal performance and VM isolation.

Future perspectives:

- Enhanced kernel-level security mechanisms
- Better orchestration and automation (Kubernetes, etc.)
- Integration with AI-driven workload management and emerging edge-computing environments

8. Personal Commentary (Relation to Your Topic)

This article opens valuable perspectives if your topic involves **cloud computing, DevOps, or container orchestration**. Understanding how namespaces and cgroups operate provides a strong foundation for:

- Building efficient containerized infrastructures
- Developing secure and resource-optimized deployments
- Contributing to tools or frameworks enhancing isolation and scalability in distributed systems

It also suggests opportunities for exploring **hybrid architectures** that combine container performance with enhanced isolation — a relevant path for future research or professional projects in DevOps and system design.

Testing the functioning of the voluntary application and defining the tasks to be done

References

ref: [linux namespaces doc](#)

[man documentation](#)